

STRING SLICING IN PYTHON



PYTHON



NOTES:

- Slicing **always** returns a new string, even if the slice includes only **one character** or **none at all**.
- If **'start'** is greater than or equal to **'end'**, an empty string is returned.
- Python **handles out-of-range indices gracefully** by adjusting them to fit within the bounds of the string.

String slicing is a **powerful feature in Python** that makes it **easy to work** with **substrings** and **manipulate text data** **efficiently**.



EXAMPLES OF STRING SLICING

1. Basic Slicing:

```
s = "Hello, World!"  
print(s[7:]) # Output: "World!"  
print(s[:5]) # Output: "Hello"  
print(s[7:12]) # Output: "World"
```

2. Using Negative Indices:

```
s = "Hello, World!"  
print(s[-6:]) # Output: "World!"  
print(s[:-8]) # Output: "Hello"  
print(s[-6:-1]) # Output: "World"
```

3. Slicing with a Step:

```
s = "Hello, World!"  
print(s[::2]) # Output: "Hlo ol!"  
print(s[1::2]) # Output: "el,Wrd"
```

4. Reverse a String:

```
s = "Hello, World!"  
print(s[::-1]) # Output: "!dlroW  
,olleH"
```



STRING SLICING IN PYTHON

In Python, string slicing **allows you to extract a portion** of a string, **creating a new string** from the original one.

The syntax for string slicing is:

```
string_name[start index : end index : step]
```

Here's what each part of the syntax means:

- **string_name**: The original string you want to slice.
- **start**: The starting index of the slice (inclusive). If omitted, it defaults to 0 (beginning of the string).
- **end**: The ending index of the slice (exclusive). If omitted, it defaults to the end of the string.
- **step**: The step or increment between each index. If omitted, it defaults to 1.

